

BedLog Raspberry Pi Cloning Guide

Master-image procedure for duplicating a fully configured BedLog Raspberry Pi installation to additional Raspberry Pi devices.

Important: This process makes a complete disk clone. It includes Raspberry Pi OS, PostgreSQL, Node/npm, BedLog files, systemd services, Chromium kiosk configuration, display settings, and anything else stored on the source microSD card.

1. Prepare the master Raspberry Pi

Before imaging, verify that the master Pi is in the state you want every clone to inherit. For a clean deployment image, BedLog should be at the onboarding screen, PostgreSQL application tables should be empty, and the BedLog services should be enabled.

```
systemctl is-active postgresql
systemctl is-active bedlog-backend
systemctl is-active bedlog-vite
systemctl is-active bedlog-kiosk

curl -s http://localhost:4002/health
curl -s -o /dev/null -w "Frontend HTTP %{http_code}\n" http://localhost:5173
```

2. Clean the Chromium BedLog profile

If the master Pi has already been onboarded, reset the dedicated Chromium kiosk profile before creating the master image. This removes the browser localStorage containing the BedLog configuration and returns the application to fresh onboarding.

```
sudo systemctl stop bedlog-kiosk.service

mv /home/pi/.config/bedlog-chromium \
  /home/pi/.config/bedlog-chromium-backup

sudo systemctl start bedlog-kiosk.service
```

3. Optionally clear BedLog records from PostgreSQL

For a clean master image, remove device-specific BedLog records from PostgreSQL. This keeps the database schema while removing data belonging to the master device.

```
PGPASSWORD=pi psql -h localhost -U bedlog -d bedlog

BEGIN;
DELETE FROM patients;
DELETE FROM beds;
DELETE FROM wards;
DELETE FROM backup_log;
DELETE FROM device_config;
COMMIT;

\q
```

4. Shut down the master Pi

Shut the Raspberry Pi down cleanly before removing its microSD card. This is especially important because PostgreSQL may otherwise have pending disk writes.

```
sudo poweroff
```

5. Identify the master microSD card on the Mac

Insert the master microSD card into the Mac and identify the external physical disk. In the installation used for this guide, the 63.9 GB Raspberry Pi card appeared as /dev/disk4. Always verify the identifier again because macOS can assign a different disk number after reconnecting devices.

```
diskutil list
```

6. Unmount the master card

Unmount the entire disk. Do not eject it yet because dd still needs access to the raw device.

```
diskutil unmountDisk /dev/disk4
```

7. Create the BedLog master image

Read the entire master card into an image file on the Mac. Use the raw rdisk device for better performance. Replace the disk number if your SD card is not disk4.

```
sudo dd if=/dev/rdisk4 \
  of=/Users/wealth/Desktop/BedLog-Master.img \
  bs=4m
```

8. Monitor and verify image creation

While dd is running on macOS, press Ctrl+T to display progress. Do not press Ctrl+C. After dd completes, flush pending writes and verify the image file. The source used for this deployment produced a roughly 59 GB image from a 63.9 GB card.

```
sync
ls -lh /Users/wealth/Desktop/BedLog-Master.img
```

9. Eject and preserve the master card

Once the image has been created and verified, safely eject the original master microSD card and keep it as a known-good recovery copy.

```
diskutil eject /dev/disk4
```

10. Insert and identify a destination card

Insert a new microSD card. Run diskutil list again and identify the external physical device. Do not assume it will use the same disk number. The destination card must be at least as large in actual byte capacity as the source image.

```
diskutil list
```

11. Unmount the destination card

After positively identifying the destination disk, unmount it. The next step permanently overwrites the destination card.

```
diskutil unmountDisk /dev/disk4
```

12. Write the BedLog master image to the new card

Write the master image to the destination raw disk. This erases all existing partitions and files on the destination card.

```
sudo dd \
  if=/Users/wealth/Desktop/BedLog-Master.img \
  of=/dev/rdisk4 \
  bs=4m
```

13. Finish and eject the cloned card

Wait until dd has completely returned to the shell prompt. Then flush writes and eject the card.

```
sync
diskutil eject /dev/disk4
```

14. First boot of each cloned Raspberry Pi

Insert the cloned card into the new Raspberry Pi and boot it. Because this is a complete disk clone, PostgreSQL, BedLog backend, Vite frontend, Chromium kiosk, LightDM/labwc setup, landscape display configuration, and systemd services should already be present.

```
hostname

systemctl is-active postgresql
systemctl is-active bedlog-backend
systemctl is-active bedlog-vite
systemctl is-active bedlog-kiosk

curl -s http://localhost:4002/health
```

15. Give each Raspberry Pi a unique hostname

Every clone initially inherits the master's hostname. Change it so each unit can be identified independently. Examples are BedLog3, BedLog4, BedLog5, and so on.

```
sudo hostnamectl hostname BedLog3
sudo reboot
```

```
# After reboot:
hostname
```

16. Confirm fresh BedLog onboarding

Before assigning the new Pi to a ward, confirm that BedLog opens at the onboarding/setup screen rather than showing the master Pi's previous hospital or ward. If it still contains the old browser state, stop the kiosk and reset the dedicated Chromium profile again.

```
sudo systemctl stop bedlog-kiosk.service
mv /home/pi/.config/bedlog-chromium \
  /home/pi/.config/bedlog-chromium-old
sudo systemctl start bedlog-kiosk.service
```

17. Verify PostgreSQL on the clone

After onboarding and using BedLog, verify that the local PostgreSQL backup is receiving records.

```
PGPASSWORD=pi psql -h localhost -U bedlog -d bedlog

SELECT 'device_config' AS table_name, COUNT(*) FROM device_config
UNION ALL
SELECT 'wards', COUNT(*) FROM wards
UNION ALL
SELECT 'beds', COUNT(*) FROM beds
UNION ALL
SELECT 'patients', COUNT(*) FROM patients
UNION ALL
SELECT 'backup_log', COUNT(*) FROM backup_log;

\q
```

Deployment Checklist

Check	Expected result
Master browser	Fresh BedLog onboarding screen
Master PostgreSQL data	Device-specific application tables empty
Master image	BedLog-Master.img created and verified
Destination card	Equal or larger actual capacity than source
Hostname	Unique on every Raspberry Pi
PostgreSQL	Active
bedlog-backend	Active and enabled
bedlog-vite	Active and enabled
bedlog-kiosk	Active and enabled
Backend health	{"ok":true}
Frontend	HTTP 200
Display	Landscape as configured
BedLog identity	Unique onboarding/device configuration per Pi

Safety note: Before every dd write operation, run `diskutil list` and positively identify the external destination card. Writing to the wrong disk can permanently destroy data on that disk.